CLAUDE.md

本文件为 Claude Code (claude.ai/code) 提供项目代码指引。

文档管理规范

> **PRD 优先于代码。** 这是本项目的最高优先级规则。任何代码改动（包括 Bug 修复）都必须先有 PRD。

所有项目文档位于 `.project/` 下，遵循 vibecoding-doc-standard.md。Agent **必须**遵守以下铁律：

1. **无 PRD 拒绝工作**：在每一次的需求开发和bug修改，没有需求文档不写代码，Bug 修复也不例外。这一点一定要记住！！！因为如果没有prd，将来代码开发将无法追溯和管理，这非常可怕！！！
2. **修改必留痕**：任何文档改动追加变更行
3. **API 必须最新**：接口改了文档必须立即同步
4. **合并必解冲突**：合并代码时同步解决文档差异
5. **先 PRD 后代码**：Agent 必须在写任何代码之前先确认 PRD 已存在或创建 PRD
6. **写 PRD 操作要独立**：Agent 在写 PRD 时要创建一个subagent去写，主agent在后续开发时去读取这个文件
7. **中文书写**: Agent在写 PRD 时，使用中文书写

其他规则：

- 每个模块自包含：功能设计、changelog、bugfix 在 `modules/<name>/features/<feature>/` 下
- PRD 按层级分目录：`prd/project/`（项目级）、`prd/module/<name>/`（模块级）、`prd/feature/<name>/`（功能级）、`prd/bugfix/<name>/`（Bug 修复级）
- 指令人必确认：创建/大改文档问用户
- 版本命名带描述：`<名称>-vN`
- Bug 记在对应功能的 `bugfix.md`，同时写 `prd/bugfix/<module>/bugfix-<简述>-vN.md`
- 全局变更记在 `.project/changelog/CHANGELOG.md`
- **跨模块逐功能更新 changelog**：一个 PRD 影响多个功能时，必须为每个受影响功能的 `changelog.md` 追加条目，不能只更新全局 CHANGELOG
- **架构文档与模块目录同步**：新增/删除模块时，`architecture/` 的模块划分表和全景图必须同步更新
- **模块文档标注代码归属**：「内部组件」表标注文件路径，「归属 Hooks」「归属 IPC Handler」段标注逻辑上属于本模块但物理平铺的文件
- **基础设施不建独立文档**：analytics、perf、notify-channels 等体量小或辅助性的代码区域，归属到相关业务模块下作为功能，不单独建模块
- **模块删除同步清理代码**：删除模块文档时，必须同步清理对应代码（service/controller/hooks/store/IPC），不能只删文档留代码

编码规范

所有代码修改必须遵循 Development-Standards-Guide.md。 核心规则：

- TypeScript strict 模式，禁止 `any`（用 `unknown`），纯类型导入使用 `import type`
- IPC 通道名必须使用 `src/shared/constants/` 中的常量，禁止硬编码字符串

- Preload 禁止暴露原始 `ipcRenderer`
- 所有 IPC handler 必须有 try/catch，返回 `{ success, data/error }` 结构
- React：只允许函数组件，Zustand 按功能拆分 store
- 命名：文件夹 kebab-case，组件 PascalCase，接口不加 `I` 前缀
- 提交前运行 `npm run lint:fix`；pre-commit hooks 会自动处理
- **编辑文件后必须运行** `npx eslint --fix <file>`：项目强制 LF 行尾（`.prettierrc` + `.gitattributes`），Windows 环境下 Prettier 会全文件重写行尾。编辑后必须立即执行 lint:fix 并**重新 Read 被修改的文件确认逻辑改动未被覆盖**，尤其是 `eslint --fix` 输出大量 `Delete ␍` 时

项目概述

AICO-Bot 是一个开源 Electron 桌面应用，将 Claude Code 的 AI Agent 能力封装为可视化跨平台界面。用户无需使用终端即可与 AI Agent 交互。2.x 版本包含 Digital Humans 自动化平台。

构建/测试命令

```
# 开发
npm run dev              # 启动开发服务器（使用 ~/.aico-bot-dev 数据目录，端口 8081）

# 构建
npm run build            # 构建 proxy + electron-vite build（输出到 out/）
npm run build:mac        # 构建 macOS universal（dmg + zip）
npm run build:win        # 构建 Windows x64（nsis 安装包）
npm run build:linux      # 构建 Linux x64（AppImage）

# 国际化（提交新用户可见文本前运行）
npm run i18n             # 提取 + 翻译
npm run i18n:extract     # 仅提取 key
npm run i18n:translate   # 仅 AI 翻译

# 二进制准备
npm run prepare          # 下载当前平台的二进制文件
npm run prepare:all      # 下载所有平台的二进制文件

# 发布
npm run release:win      # 构建并发布 Windows 版本到 GitHub Releases（需 GH_TOKEN）
npm run release          # 同时构建并发布多平台版本

# 代码质量
npm run typecheck        # TypeScript 类型检查
npm run lint             # ESLint 检查
npm run lint:fix         # ESLint 自动修复
npm run format           # Prettier 格式化
```

Windows 打包流程

详见 `docs/WINDOWS_DEV.md`。核心步骤：

1. **环境准备**：安装 Node.js 20.x、Python 3.x、Visual Studio Build Tools 2022+
2. **安装依赖**：`npm install`（会自动执行 postinstall 应用补丁）
3. **下载二进制**：`npm run prepare`（下载 cloudflared、gh CLI、better-sqlite3 prebuild 等）
4. **构建打包**：`npm run build:win`（构建 proxy → electron-vite build → electron-builder 打包 NSIS 安装包）
5. **发布**（可选）：在 `.env.local` 配置 `GH_TOKEN`，然后 `npm run release:win`

构建产物：

- `out/` — electron-vite 编译输出
- `dist/` — electron-builder 打包输出（`AICO-Bot Setup x.x.x.exe`）

常见问题：

- 原生模块编译失败 → `npm config set msvs_version 2022 && npm install`
- better-sqlite3/node-pty 报错 → `npx electron-rebuild && npm run prepare`
- `npm run prepare` 下载失败 → 设置 `HTTPS_PROXY` 或手动下载

架构概述

多进程 Electron 架构

```
┌──────────────────────────────────────────────────────────────────────┐
│                        AICO-Bot 桌面端                                 │
│  ┌──────────────┐     ┌──────────────┐     ┌──────────────────┐      │
│  │   React UI   │◄──►│    主进程    │◄──►│  Claude Code SDK  │      │
│  │ (渲染进程)   │IPC │   (Main)     │     │  (Agent 循环)     │     │
│  └──────────────┘     └──────────────┘     └──────────────────┘      │
│                              │                                        │
│                              ▼                                        │
│                      ┌──────────────┐                                 │
│                      │  本地文件    │                               │
│                      │ ~/.aico-bot/ │                                 │
│                      └──────────────┘                                 │
└──────────────────────────────────────────────────────────────────────┘
```

应用通过 `src/main/bootstrap/` 分两阶段启动：

- **阶段一（Essential）**：同步初始化，目标 <500ms — 空间、配置、IPC 处理器
- **阶段二（Extended）**：延迟到 `ready-to-show` 之后 — 分析、健康检查、后台服务

关键目录

- `/src/main/` — Electron 主进程
  - `services/agent/` — 核心 Agent，SDK 集成，消息处理
  - `services/remote-ws/` — 远程服务器 WebSocket 客户端
  - `services/remote-ssh/` — SSH 隧道
  - `services/remote-deploy/` — 远程 Agent 部署
  - `services/space.service.ts` — 工作空间管理
  - `services/conversation.service.ts` — 聊天记录持久化
  - `platform/` — 后台子系统（事件总线、记忆、调度、后台应用）
  - `apps/` — Digital Humans 自动化平台（规范、管理、运行时）
  - `ipc/` — IPC 处理器模块
  - `http/` — 远程访问 HTTP 服务
- `/src/renderer/` — React 前端
  - `components/` — UI 组件
  - `stores/` — Zustand 状态管理（`chat.store.ts` 是最核心的）
  - `api/` — 双模式 IPC/HTTP 适配层（见下方说明）
  - `pages/` — 页面组件
  - `i18n/` — 国际化（7 种语言）
- `/src/shared/` — 前后端共享（禁止导入 Node.js 或 Electron 模块）
  - `types/` — 共享类型定义
  - `constants/` — 共享常量
- `/src/preload/` — Preload 脚本，暴露 `window.aicoBot`
- `/src/worker/` — 文件监听器（作为独立子进程运行）
- `/packages/remote-agent-proxy/` — 独立 Node.js 服务，用于远程 Claude 访问

双模式渲染器 API

渲染器 API 层（`src/renderer/api/`）以两种模式工作：

- **Electron 模式**：方法通过 IPC preload 桥调用 `window.aicoBot.xxx()`
- **远程 Web 模式**：方法调用 HTTP 端点 + WebSocket 事件

`transport.ts` 通过 `isElectron()`（检查 `window.aicoBot`）自动检测模式。`api/index.ts` 导出统一的 `api` 对象。

添加 IPC 端点

添加新 IPC 通道时，更新以下 3 个文件：

1. `src/preload/index.ts` — 暴露到 `window.aicoBot`
2. `src/renderer/api/transport.ts` — 在 `onEvent()` 中添加到 `methodMap`
3. `src/renderer/api/index.ts` — 导出为 `api.xxx`

路径别名

- `@/` → `src/renderer/`（渲染进程代码）
- `@main/` → `src/main/`（测试用）
- `@shared` → `src/shared/`（测试用）

远程 Agent 架构

远程空间通过 WebSocket 代理路由：

1. **SSH 隧道**（可选）：`sshTunnelService.establishTunnel()` 创建本地端口转发
2. **WebSocket 客户端**：`RemoteWsClient` 连接远程 Agent
3. **会话恢复**：`sdkSessionId` 实现多轮对话

关键配置：

- `space.claudeSource === 'remote'` 触发远程执行
- `space.remoteServerId` 标识目标服务器
- `space.useSshTunnel`（默认：true）决定连接模式

远程空间思考过程流

```
远程服务器 (stream-processor.ts)    本地客户端 (send-message.ts)    前端 (chat.store.ts)
────────────────────────────────    ────────────────────────────    ────────────────────
SDK stream_event                    RemoteWsClient 事件             Zustand store 更新
├─ thinking start ──────────────▶ 'thought' 事件 ──────────────▶ handleAgentThought()
├─ thinking delta ──────────────▶ 'thought:delta' 事件 ──────────▶ handleAgentThoughtDelta()
└─ thinking stop ───────────────▶ (完成信号) ────────────────────▶ thought.isStreaming =
false
```

代码约定

提交格式

使用 conventional commits（中文描述）：`feat:`、`fix:`、`docs:`、`style:`、`refactor:`、`chore:`

示例：`feat(ipc): 添加 IPC 通道常量定义`、`fix(chat): 修复重连后消息不显示`

UI 禁止硬编码文本

所有用户可见字符串必须使用 `t()`：

```
// 正确
<Button>{t('Save')}</Button>

// 错误 — 硬编码文本会破坏国际化
<Button>Save</Button>
```

英文是源语言 — `t('English text')` 本身就是英文值。提交新用户可见文本前运行 `npm run i18n`。

Tailwind 样式

使用 CSS 变量主题色，禁止硬编码值：

```
// 正确
<div className="bg-background text-foreground border-border">

// 错误
<div className="bg-white text-black border-gray-200">
```

状态管理

- **Zustand** 用于前端状态（参见 `chat.store.ts`）
- 每会话状态：`Map<conversationId, SessionState>`
- 每空间状态：`Map<spaceId, SpaceState>`

IPC 通信

- 主进程 → 渲染进程：`sendToRenderer('agent:event', spaceId, conversationId, data)`
- 渲染进程 → 主进程：使用 `src/renderer/api/` 中的 `api.*` 方法

SDK 集成

使用 `@anthropic-ai/claude-agent-sdk` V2 会话（`src/main/services/agent/session-manager.ts`）：

- `unstable_v2_createSession()` — 创建新会话
- `unstable_v2_resumeSession()` — 恢复已有会话

关键 SDK 选项（`src/main/services/agent/sdk-config.ts`）

```
{
  model: credentials.sdkModel,          // 从配置动态读取，不硬编码
  cwd: workDir,
  permissionMode: 'bypassPermissions',
  includePartialMessages: true,
}
```

环境变量

将 `.env.example` 复制为 `.env.local`。关键变量：

- `AICO_BOT_TEST_API_KEY`、`AICO_BOT_TEST_API_URL`、`AICO_BOT_TEST_MODEL`、`AICO_BOT_TEST_PROVIDER` — i18n 翻译
- `GH_TOKEN` — 发布 Release
- 分析变量（`AICO_BOT_GA_*`、`AICO_BOT_BAIDU_*`）— 可选，默认关闭

配置文件

- `product.json` — 构建配置和认证 Provider 定义
- `~/.aico-bot/` — 用户数据目录（对话、设置、空间）
- `~/.aico-bot-dev/` — 开发模式数据目录（`npm run dev` 使用）
- `.env.local` — 本地环境变量覆盖（已 gitignore）
- `electron.vite.config.ts` — 构建配置（main/preload/renderer 入口）